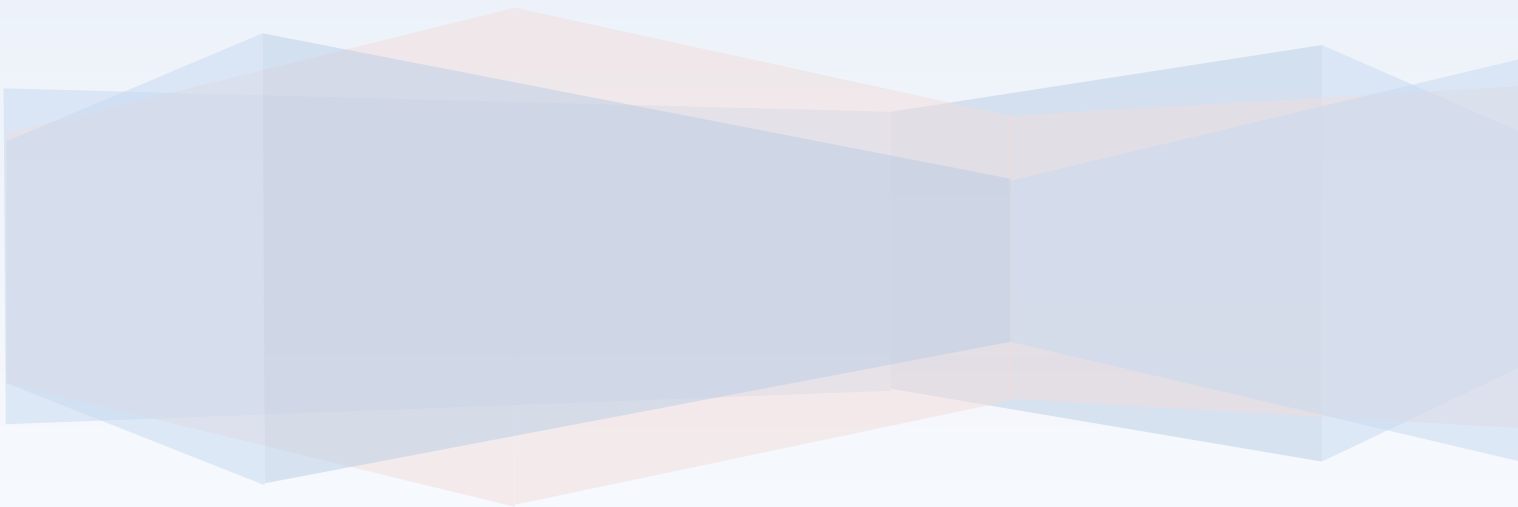


COS30002 Artificial Intelligence for Games

Semester 1, 2026

Learning Summary Report

Pattarapol Tantechasa (103883220)



Declaration

I declare that this portfolio is my individual work. I have not copied from any other student's work or from any other source except where due acknowledgment is made explicitly in the text, nor has any part of this submission been written for me by another person or software service.

Signature: Pattarapol Tantechasa

Self-Assessment Details

The following checklists provide an overview of my self-assessment for this unit.

	Pass (P)	Credit (C)	Distinction (D)	High Distinction (Low HD) (High HD)	
Self-Assessment (please tick)		✓			

Self-assessment Statement

	Included? (tick)
Learning Summary Report	✓
Complete Pass ("core") task work, approved in Canvas	✓

Minimum Pass Checklist

Introduction

This report summarises what I learnt in COS30002 AI for games. It includes a self-assessment against the criteria described in the unit outline, a justification of the pieces included, details of the coverage of the unit intended learning outcomes, and a reflection on my learning.

- **Task 1 Lab, BitBucket Setup:** Created and configured a private BitBucket repository, added .gitignore and README, invited tutor with read-only access, and documented the setup with screenshots. Establishes version control foundation used across all subsequent tasks.
- **Task 2 Lab, FSM & Python:** Designed a 3-state FSM (Resting, Gathering, Crafting) with an FSM diagram and working Python implementation. States transition based on energy and wood thresholds, demonstrating how state machines model agent behaviour in games.
- **Task 3 Lab, Tic-Tac-Toe:** Built a Pygame Tic-Tac-Toe game using OOP with two AI agents, RandomAI (random moves) and SmartAI (win → block → random priority). Demonstrates goal-oriented behaviour and the standard game loop pattern (process_input, update_model, render).
- **Task 4 Spike, Graphs, Search & Rules:** Extended Tic-Tac-Toe to model game states as a graph. Implemented three AI agents — RandomSearchAI (DFS random walk), EfficientSearchAI (adds visited-state deduplication), and MinimaxAI (exhaustive optimal search that never loses). Extension added a search tree visualiser printing the full winning path to console.
- **Task 5 Lab, Graphs, Paths & Search:** Implemented and compared DFS, BFS, Dijkstra, and A* on a terrain grid with varied movement costs. Added diagonal movement, custom heuristics, new terrain types (forest, hill), custom maps, and an animated agent. Fixed an A* admissibility bug caused by incorrect min_edge_cost value.
- **Task 6 Spike, Navigation with Graphs:** Extended Task 5 into an interactive navigation tool using Pyglet with real-time terrain editing, step-by-step algorithm execution, and toggleable views. Fixed multiple bugs including enum casting errors and float evaluation crashes.
- **Task 7 Lab, Goal Oriented Behaviour & SGI:** Implemented Simple Goal Insistence (SGI) in two scripts — a working case (2 independent goals satisfied in 3 steps) and a deliberate failure case (interdependent goals causing infinite oscillation). Demonstrates both the capability and key limitations of greedy planning.
- **Task 8 Spike, Goal-Oriented Action Planning (GOAP):** Built a complete GOAP system for a wizard agent: world state, 11 actions with preconditions and effects, and an A* planner. Produced different optimal plans (7 steps vs 16 steps) automatically when world conditions changed, with no code modification required.
- **Task 9 Lab, PlanetWars:** Implemented six PlanetWars bots of increasing sophistication — Rando, BestWorst, Defender (threat detection + reactive reinforcement), Scout (vision_age-based reconnaissance), and Closest (proximity targeting). Demonstrates tactical target selection, fleet management, and information-driven decision-making.
- **Task 10 Spike, Tactical Analysis with PlanetWars:** Built TacticalBot (garrison + affordability + growth/proximity scoring) and validated it across 100 games — 97% win rate. Extended with asymmetric map design (TacticalBot won 100% from both sides) and FogBot (stale-data ship estimation + active scouting).

Reflection

The most important things I learnt:

- What a different AI decision-making approaches each solve different classes of problems. FSM (Task 2) handles predictable state-driven behaviour, GOB (Task 3) handles priority-based reactive decisions, and GOAP (Task 8) handles multi-step planning where actions have side effects. Seeing all three back-to-back made it clear that no single approach fits every problem.
- Graph search algorithms have distinct trade-offs, and choosing the wrong one has real costs. DFS, BFS, Dijkstra, and A* were all implemented and compared in Task 5 (Lab) and Task 6 (Spike). DFS outperformed A* in a maze, which was counterintuitive and highlighted that algorithm choice depends on environment structure, not just theory.
- Simple, structured tactical rules can produce dominant AI behaviour, in Task 10 (Spike), TacticalBot using just three rules (garrison, affordability, growth+proximity scoring) beat NaiveBot 97 out of 100 games. This showed that well-designed simple logic outperforms complex random behaviour.
- Planning algorithms like GOAP (Task 8 Spike) solve problems that greedy SGI cannot. The SGI failure case in Task 7 (Lab) showed how a one-step greedy planner oscillates endlessly, while GOAP's A* search over world states finds a stable optimal plan by considering future consequences.

The things that helped me most were:

- Visualising agent behaviour using Pygame (Tasks 2, 3, 4) and Pyglet (Tasks 5, 6, 9, 10), watching agents move and make decisions in real time made it immediately obvious when something was wrong, far faster than reading console output alone.
- Building deliberate failure cases alongside working ones in Task 7 (Lab), implementing both the working SGI and the oscillating failure side-by-side made it concrete why the greedy approach breaks down, rather than just being told it does.
- Incremental task structure, each assignment directly extended the previous one. Task 4 extended Task 3's Tic-Tac-Toe with graph search; Task 8 extended Task 7's GOB with full planning. This meant each new concept had an existing foundation to attach to.

I found the following topics particularly challenging:

- **GOAP action precondition and effect design (Task 8 Spike)**, defining 11 actions with correct preconditions, effects, and costs so the planner could sequence them into a valid plan took significant iteration. Small errors in world state transitions caused the planner to either fail or find suboptimal paths.
- **Fog-of-war estimation (Task 10 Spike)**. TacticalBot used stale vision data as if it were current, leading to failed attacks on planets that had grown since last observation. Building FogBot required shifting from treating game state as ground truth to treating it as an estimate that degrades over time.

I found the following topics particularly interesting:

- **Minimax in Task 4 (Spike)**, the idea that an agent searching the complete game tree can be made mathematically unbeatable. Implementing it alongside RandomSearchAI and EfficientSearchAI made the progression from blind search to optimal search very tangible.
- **Asymmetric map results in Task 10 (Spike)**, a hand-crafted map gave one player nearby planets with total growth 14 vs the other side's 5. TacticalBot still won 100% of games from the weaker side, because NaiveBot could not exploit its own advantage. This was a striking demonstration that strategic reasoning matters more than starting position.
- **SGI failure case in Task 7 (Lab)**, watching the agent oscillate infinitely between "drink coffee" and "eat heavy meal" because each action undid the other was both clear and memorable. It made the need for proper planning feel necessary rather than theoretical.

I feel I learnt these topics, concepts, and/or tools really well:

- **Pathfinding algorithms (Task 5 Lab, Task 6 Spike)** — implemented DFS, BFS, Dijkstra, and A* from scratch on a terrain grid with multiple cost types. Added diagonal movement, custom heuristics, new terrain types (forest, hill), and custom maps. Diagnosed and fixed the admissibility bug. The interactive visualisation in Task 6 allowed step-by-step comparison of all four algorithms on any terrain layout.
- **GOAP planning (Task 8 Spike)** — designed and built a complete GOAP system: world state representation, 11 action definitions with preconditions and effects, A* planner, and simulation runner. The system automatically produced different plans (7 steps vs 16 steps) when the available quest changed, without any code modification. This is evidence that the planning architecture was correctly separated from the domain.
- **PlanetWars AI strategy (Task 9 Lab, Task 10 Spike)** — implemented six bots of increasing sophistication in Task 9, then in Task 10 built TacticalBot with multi-criteria scoring, validated it across 100 games, designed asymmetric maps to test positional bias, and extended to FogBot with stale-data compensation and active scouting.

I still need to work on the following areas:

- **Steering force models (ULO3)**, force-based movement techniques were not covered in any assignment task but rather portfolio tasks later. Tasks 5 and 6 showed path-following (an agent walking a computed path), but this is driven by pathfinding output, not velocity and force accumulation. This remains a gap in my learning.
- **Deeper theoretical analysis**, the reports and release notes focused on describing what was implemented and what results were observed. In future work I would go further into algorithmic time

and space complexity and explain more rigorously why one approach outperforms another beyond empirical results.

My progress in this unit was ...:

- Submitted all 10 tasks across 5 assignments on time and engaged with the tutor for weekly checkoffs throughout the unit.
- Progress was consistent week-to-week — because each assignment built on the previous one, keeping up with submissions directly supported the quality of later tasks. Falling behind on Task 5 would have made Task 6 and GOAP much harder.
- The steady pace meant no last-minute cramming, and extensions (asymmetric maps, FogBot, custom terrain) were only possible because the baseline tasks were completed and understood before the deadline arrived.

This unit will help me in the future:

- Graph search and planning algorithms (Tasks 4, 5, 6, 8) are foundational across game AI, robotics, and any domain involving optimal decision-making under constraints. These are directly reusable skills.
- The GOAP architecture (Task 8) and behaviour-tree-adjacent patterns from GOB (Tasks 3, 7) are industry-standard approaches used in commercial game development. Having implemented them from scratch gives a concrete foundation for building my own indie game that planned to work on in the future.

If I did this unit again, I would do the following things differently:

- Add a visualisation layer earlier — even a basic one. Attaching a simple GUI to the FSM in Task 2 would have made state transitions easier to trace and debug and would have built Pygame familiarity before Task 3 required it.
- Write short reflection notes at the end of each task instead of consolidating everything at the end. Specific details like which bug was hardest to fix and what was surprising are much clearer in the moment than weeks later.

Conclusion

In summary, I believe that I have clearly demonstrate that my submission is sufficient to be awarded a **credit** grade.

- Completed all 10 tasks (labs & spikes) with working code and written reports across all 5 assignments.
- Demonstrated ULO1 through FSM, GOB, and GOAP implementations (Tasks 2, 3, 7, 8); ULO2 through four pathfinding algorithms with quantitative comparison (Tasks 4, 5, 6); ULO4 through adaptive planning with SGI and GOAP (Tasks 7, 8); ULO5 through layered multi-technique PlanetWars bots (Tasks 9, 10)
- Extended beyond baseline requirements in Tasks 4, 5, 6, 9, and 10 — Task 4 added a search tree visualiser (render_board and print_path on GameNode) so the full winning path is printed step-by-step to console; Tasks 5 and 6 added custom terrain types and interactive visualisation; Tasks 9 and 10 added additional bot strategies, asymmetric map design, and fog-of-war handling
- 97% win rate in Task 10 batch testing across 100 games provides quantitative evidence that the AI techniques were not just implemented but genuinely understood and applied effectively